ITMO University

Faculty of Software Engineering and Computer Systems

Laboratory work No. 1

EXP-0 «Introductory experiment»

Course «Operating Systems»

Variant: Write, Cache, 10M vs 1M, Rand

Performed by:
Rahaman Md Afifur (Рахаман Мд Афифур)
group P3330, ISU 407793

Teacher:
Klimenkov Sergey Viktorovich (Клименков Сергей Викторович)

Saint Petersburg, 2026

Contents

1 Goal and variant . . . 3
2 Work progress . . . 3
2.1 System passport (step 4.1) . . . 3
2.2 Environment preparation (step 4.2) . . . 4
2.3 Test data and build . . . 4
2.4 Baseline observations (step 4.3) . . . 4
2.5 Hypothesis (step 4.4) . . . 5
2.6 Experiment plan (step 4.5) . . . 5
3 Results . . . 6
3.1 Stage 1: `graph_traverse` (read/lseek/write) . . . 6
3.2 Stage 2: `graph_traverse_mmap` . . . 6
3.3 Stage 3: all series together . . . 8
3.4 Intrusiveness of the measuring tools . . . 9
4 Conclusion . . . 9
5 Appendix A. Reproduction . . . 10
6 Appendix B. Removed outliers (1.5 × IQR) . . . 10
7 Appendix C. Collection script `collect.sh` . . . 10

1 Goal and variant

Goal. Experimentally compare the running time of the provided traversal programs on a modifying workload, and learn to set up an OS experiment correctly: fix and document the environment, choose tools, plan the number of measurements and process the results statistically.

Variant «Write, Cache, 10M vs 1M, Rand»: both programs run in write mode (`--write`, every visited node is updated), the system page cache is used (no `--no-cache`), the graph is a chain in random physical order (`--topology chain`), and the compared factor is the **file size**: 1 MiB vs 10 MiB. Question of the variant: **does size matter for a typical case on a modern system?**

Objects of study: `graph_traverse` (access through `lseek()` + `read()`/`write()`) and `graph_traverse_mmap` (the file is mapped into memory with `mmap()`).

2 Work progress

All scripts, raw data, logs and figures: https://github.com/simu18/os-course/tree/lab-intro-exp/lab/intro-exp (branch `lab-intro-exp`). Every number below can be reproduced with the commands in Appendix A.

2.1 System passport (step 4.1)

The experiment was run in a virtual machine. The guest sees virtual devices only, so the physical parameters were taken from the host and are marked as such.

Parameter	Value	Source
Host	MacBook Air, Apple M2: 8 cores (4 performance + 4 efficiency), 16 GB RAM, macOS 26.6.2	`system_profiler` on host
Hypervisor	UTM, QEMU backend	`systemd-detect-virt` = `qemu`
Guest OS	Ubuntu 26.04.1 LTS, kernel 7.0.0-38-generic, aarch64, PREEMPT_DYNAMIC	`uname -a`, `os-release`
vCPU	4 vCPU, 1 thread/core, 1 socket; vendor «Apple», model name hidden («-»); cache sizes not reported	`lscpu`, `lscpu -e`
Memory model	1 NUMA node, i.e. **UMA**	`lscpu`, `numactl --hardware`
RAM	4 GiB configured, 3.3 GiB usable; swap 3.8 GiB	`lshw`, `free -h`
Page size	4096 B	`getconf PAGESIZE`
Disk	virtio `vda`, 25 GB, ext4; I/O scheduler `none`; physically an image file on the host's SSD	`lsblk`, `df -hT`, host
Disk type	the guest reports `rotational = 1`, which is **not** real: virtio does not report the medium. SMART is unavailable (`Unable to detect device type`)	`/sys/block/vda/...`, `smartctl`
CPU frequency	no `cpufreq` interface in the guest: governor and Turbo cannot be read or fixed	`/sys/.../cpufreq`
PMU	hardware events (cycles, cache/TLB misses) are `<not supported>` in the guest	`perf stat`
Writeback	`dirty_ratio` 20, `dirty_background_ratio` 10, `dirty_expire_centisecs` 3000, `swappiness` 60	`sysctl`
Toolchain	Ubuntu clang 21.1.8, Python 3.14.4	`clang --version`

Table 1: System passport (full output: `logs/passport.txt`)

2.2 Environment preparation (step 4.2)

Item	Decision and reason
Background load	GUI apps closed; load average below 1 (`uptime`), all vCPUs ≈ 97 % idle (`mpstat`). Snapshots before/after each series: `logs/snapshot_*.txt`.
CPU governor, Turbo	Not controllable in the guest (no `cpufreq`); macOS also decides which physical core (P or E) runs a vCPU. Recorded as an uncontrolled factor.
Pinning	`taskset -c 2`. All vCPUs are equally idle; CPU0 additionally serves the virtio network interrupts, so it was avoided. The pinned vCPU is still scheduled by the host.
Priority	`renice -n -5` for the collection script; children inherit it.
Cache state	**Warm cache** (the variant). Fresh graphs, `sync`, then one untimed run of every configuration; caches are never dropped during a series.
Data size vs RAM	1 MiB and 10 MiB ≪ 3.3 GiB, so both files stay entirely in the page cache: 0 major page faults in all 248 final runs.
perf access	`kernel.perf_event_paranoid` = 1 set temporarily (default 4), reset on reboot.

Table 2: Environment checklist

2.3 Test data and build

```
python3 ../util/graphgen.py -s 1M  --seed 427 --topology chain -b 0.5 --min-step-pages 2 --
verify -o graphs/graph-rand-1M.bin
python3 ../util/graphgen.py -s 10M --seed 427 --topology chain -b 0.5 --min-step-pages 2 --
verify -o graphs/graph-rand-10M.bin
clang -o out/graph_traverse src/graph_traverse.c     # README flags
clang -o out/graph_traverse_mmap src/graph_traverse_mmap.c
```

Graph	Nodes	Pages (4 KiB)	Root node	sha256 (first 12)
1M	43 689	256	9 668	`6513e809b696`
10M	436 905	2 560	252 929	`9cbbf5a3804a`

Table 3: Generated graphs. Record = 24 B, header = 40 B, every jump ≥ 342 nodes (2 pages); 50.2 % forward / 49.8 % backward jumps

Generating the graphs twice gave identical sha256 sums, so generation is reproducible. Since `--write` modifies the files, the graphs were regenerated right before the final series.

2.4 Baseline observations (step 4.3)

Single runs, one pass (full table: `logs/baseline_step4.3.txt`):

Program, flags	1M wall, s	10M wall, s	vcs (10M)	Observation
traverse	0.04	0.30	1 248	first runs after file creation: warm-up effect
traverse `--no-cache`	3.02	30.50	450 474	≈ 1 disk read per node, ≈ 70 µs each
traverse `--write`	0.03	0.37	1	writes only dirty pages in RAM, no waiting
traverse `--write --no-cache`	9.10	90.51	1 326 711	3 disk ops per node (read-modify-write of a block)
mmap, any flags	≤ 0.01	0.01–0.04	1–3	below the 10 ms resolution of `/usr/bin/time`

Table 4: Baseline: one run per configuration

- **Syscalls** (`strace -c`, `graph_traverse --write 1`, 1M): 87 379 `lseek`, 43 691 `read`, 43 691 `write`, 174 793 in total, i.e. 4 syscalls per node (+1 header read, +1 libc load, +2 progress messages). The mmap version makes **37** syscalls in total and none per node.
- **Page faults of mmap** (minus ≈ 73 for program start-up): reading costs 16 faults for 1M and 162 for 10M, i.e. one fault per 16 pages (kernel **fault-around**). Writing costs 272 and 2 721 faults = one write fault per page plus the read faults: the first write to each page must mark it dirty. RSS of mmap grows with the file (2.1 → 11.4 MB); `graph_traverse` stays at 1.2 MB.
- **Warm-up**: right after generation a read of 10M took 0.30 s with 1 248 voluntary switches; the next runs took 0.17 s with 1 switch.

- **Noise**: with `stress-ng --cpu 4` the wall time of `graph_traverse --write 5` (10M) grew from 1.69–1.70 s to 2.54–2.77 s (+57 %). Involuntary context switches went from 13–17 to 180–266, and the spread grew from 0.01 s to 0.23 s.
- **perf stat**: `task-clock` ≈ elapsed (the process is CPU-bound); `cpu-migrations` 0–1 (the single migration is `taskset` moving itself to CPU 2 before `exec`); hardware events are not supported in the guest.

2.5 Hypothesis (step 4.4)

Both files fit in RAM and are fully cached, so the disk is not involved during a run. File size can therefore only act through the CPU caches and the TLB: 256 vs 2 560 pages visited in random order.

- **H1 (read/write).** The cost per node is dominated by 4 syscalls (≈ 190 ns each), which do not depend on file size. Expected: 10M at most **5 %** slower per node.
- **H2 (mmap).** There are no syscalls per node, so the cost is memory access. The larger working set fits worse into the caches and the TLB (two-stage translation in a VM). Expected: 10M **10–25 %** slower per node, and the difference shows up in **user** time.
- **H3.** mmap is more than 30× faster per node than read/write.

2.6 Experiment plan (step 4.5)

Item	Decision
Metric per run	Wall time of one process (`date +%s%N`, ns resolution), plus **ns per node** = wall / (nodes × iterations). Also recorded: user, sys, minflt, majflt, voluntary and involuntary switches (`/usr/bin/time`).
Equal work	read/write: 1M × 50, 10M × 5 passes; mmap: 1M × 2000, 10M × 200. Every run visits ≈ the same number of nodes and lasts 1.5–1.9 s; for mmap even the number of page faults is equal (544 k).
Cache	Warm, the same for every run (see 2.2).
Order	N rounds. In each round 1M and 10M run in a random order (`shuf`), and the order is recorded. Interleaving protects against drift.
Warm-up	The first **K = 2** rounds are discarded. Fixed before the final series, based on the warm-up effect observed in 2.4.
Outliers	The 1.5 × IQR rule per configuration; every removed run is listed (Appendix B).
N	Pilot N = 10 → N ≈ (t·s/E)² with E = 1 % of the mean (Table 6). Chosen **N = 60** (+ 2 warm-up rounds = 62), about 2.5× the formula's answer, as a margin for heavy tails.
Measurement	Only `/usr/bin/time` + `date` in the final series; `perf` and `strace` only for characterization.

Table 5: Experiment plan

Pilot, N = 10	mean, s	s, s	CV	N for E = 1 %	Comment
read/write 1M	1.688	0.024	1.4 %	8	
read/write 10M	1.770	0.044	2.5 %	24	
mmap 1M	1.507	0.037	2.5 %	24	
mmap 10M	1.919	0.255	13.3 %	680	two early runs (2.18, 2.56 s); ≈ 24 without them

Table 6: Pilot series and the estimated N

The final series (step 4.6–4.7) was run with `collect.sh` (Appendix C), right after a fresh `gen_graphs.sh` and `sync`, with snapshots `logs/snapshot_before_final.txt` and `logs/snapshot_after_final.txt`. Raw data: `data/final_read.csv` and `data/final_mmap.csv` (one row per run). Processing: `analyze.py`.

3 Results

3.1 Stage 1: `graph_traverse` (read/lseek/write)

Size	N	mean wall, s	95 % CI, s	±CI	median, s	ns/node	user, s	sys, s
1M × 50	56	1.7910	1.7841–1.7979	0.38 %	1.7840	**819.9**	0.363	1.414
10M × 5	55	1.9289	1.9177–1.9401	0.58 %	1.9173	**883.0**	0.366	1.551

Table 7: Stage 1 results (62 runs per size − 2 warm-up − 4/5 outliers)

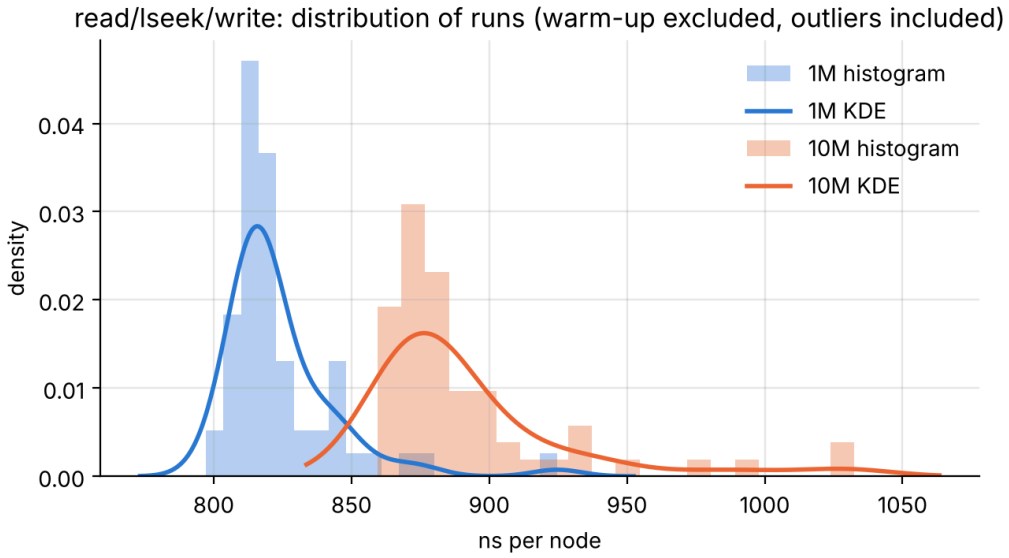


Figure 1: Stage 1: distribution of runs, histogram + KDE (warm-up excluded, outliers included)

Conclusion of stage 1 (step 4.9).

1. **Comparison.** 10M is **7.7 %** slower per node (median: 7.5 %; with outliers: 8.3 %). The Welch t-test gives t = 21.0, p ≈ 0. **H1 is not confirmed**: the effect is larger than the 5 % we predicted. The mechanism is visible in the time split: **user time is identical** (0.363 vs 0.366 s), and **the whole difference is in sys time** (+9.7 %). The program's own loop does not slow down; the kernel does, because in this method the kernel touches the file data. It copies from and to the page-cache page on every `read`/`write` and updates that page's metadata. With 2 560 pages visited at random, that data fits worse into the CPU caches than with 256 pages. Without hardware counters this explanation is supported by the user/sys split, not proven by cache-miss counts.
2. **Error.** The CI half-widths are 0.38 % and 0.58 %. The intervals do not overlap: the difference of the means (0.138 s) is ≈ 12× the larger half-width (0.011 s).
3. **Sufficiency.** The formula gives N = 9 (1M) and 19 (10M) for ±1 %; we have 56 and 55. More runs would shrink the CI as 1/√N but cannot change the conclusion.

3.2 Stage 2: `graph_traverse_mmap`

Size	N	mean wall, s	95 % CI, s	±CI	median, s	ns/node	user, s	sys, s
1M × 2000	57	1.4942	1.4821–1.5064	0.81 %	1.4723	**17.10**	1.177	0.304
10M × 200	53	1.7848	1.7663–1.8033	1.04 %	1.7546	**20.43**	1.448	0.323

Table 8: Stage 2 results (62 runs per size − 2 warm-up − 3/7 outliers)

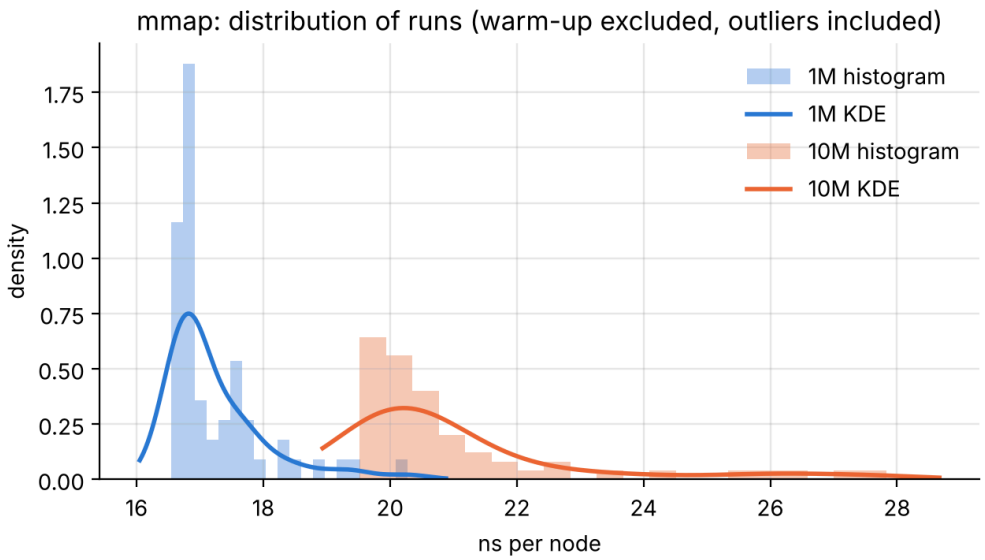


Figure 2: Stage 2: distribution of runs, histogram + KDE

Per run (final series)	read/write 1M	read/write 10M	mmap 1M	mmap 10M
minor page faults	137	137	544 149	544 265
major page faults (all runs)	0	0	0	0
voluntary switches	1.02	1.04	1.35	1.17
involuntary switches	14.6	16.3	12.3	14.5
syscalls per pass	≈ 174 800	≈ 1 747 600	≈ 7	≈ 7

Table 9: Context switches and page faults: read/write vs mmap

Conclusion of stage 2.

1. **Comparison.** 10M is **19.4 %** slower per node (median 19.2 %, with outliers 22.2 %; t = 26.4, p ≈ 0). **H2 is confirmed.** Here the difference is in **user** time (+23 %), while sys time (+6 %) and the number of page faults are practically equal by design. The program itself touches the mapped page, so the cache/TLB penalty of the larger working set appears in user space.
2. **Page faults vs context switches.** read/write has ≈ 137 faults per run (program start-up only) and ≈ 175 k syscalls per pass. mmap has no per-node syscalls, but pays one fault per touched page per pass: 272 for 1M (256 write faults + 16 read faults with fault-around) and 2 721 for 10M. Handling 544 k faults costs ≈ 0.30 s of sys time per run, ≈ 0.56 µs per fault. Voluntary switches stay ≈ 1 in both programs: nothing waits for the disk.
3. **Error and sufficiency.** The CIs (±0.81 % and ±1.04 %) do not overlap. The formula gives N = 38 and 57 for ±1 %; we have 57 and 53, so mmap 10M is at the target. The difference of the means (0.291 s) is ≈ 16× the larger half-width, so more runs would not change the conclusion.
4. **H3 is confirmed**: mmap is **48×** (1M) and **43×** (10M) faster per node than read/write.

The conclusions differ from stage 1, as the task predicts: with syscalls the size effect is hidden behind a large fixed cost and appears in the kernel; with direct memory access it is 2.5× larger in relative terms and appears in the program itself. In absolute terms read/write loses more per node (+63 ns vs +3.3 ns), because every node is touched twice by kernel copies.

3.3 Stage 3: all series together

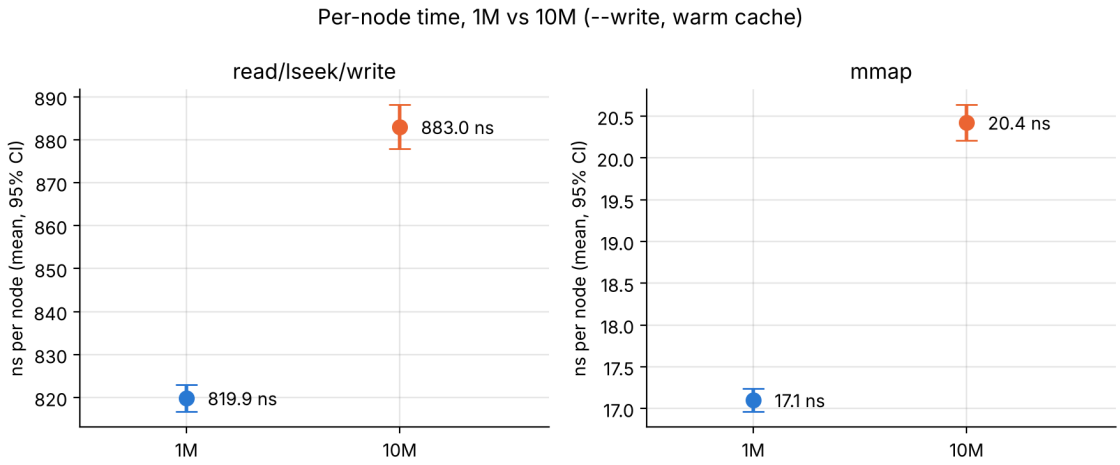


Figure 3: Mean time per node with 95 % confidence intervals

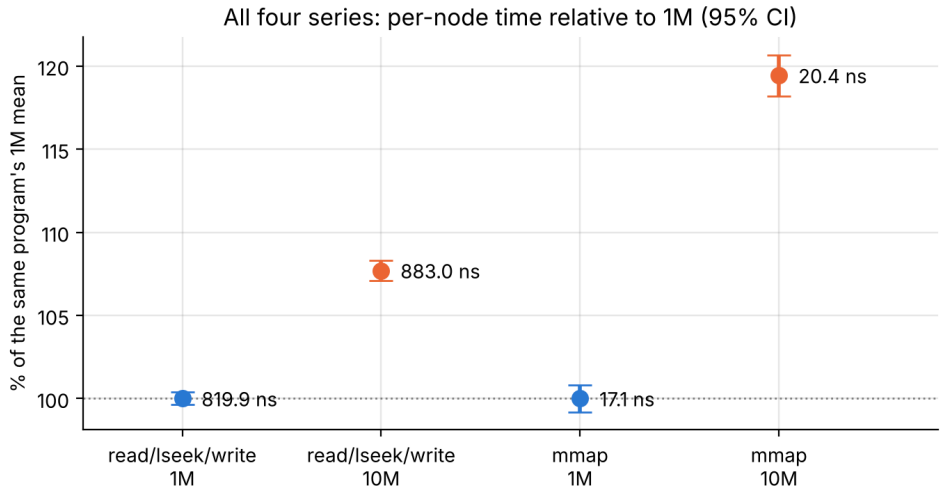


Figure 4: All four series relative to the 1M mean of the same program

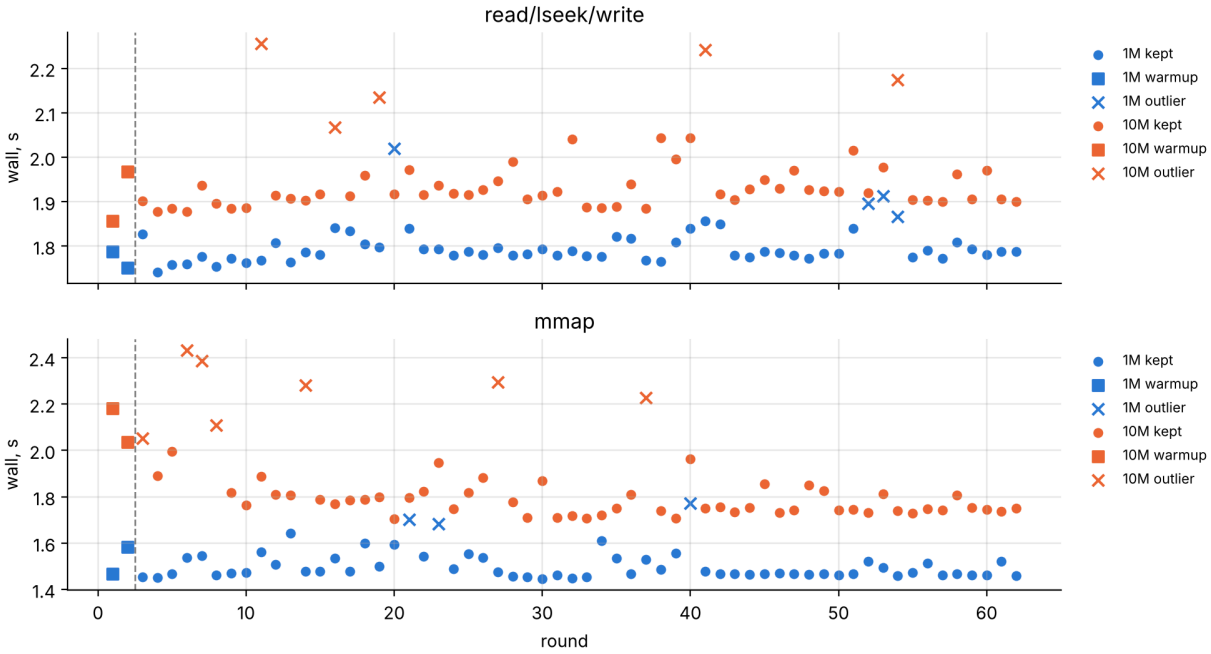


Figure 5: Every run in execution order: warm-up (squares), outliers (crosses)

The distributions are not normal: Shapiro–Wilk $p < 0.001$ for all four series, with a long **right** tail of rare slow runs. That is why the median and the IQR are reported next to the mean, and why the conclusions were also checked with outliers included. Outlier runs have both user **and** sys inflated together, and sometimes hit neighbouring rounds of both sizes at once (read/write rounds 52–54, mmap rounds 35–40). That is a slowdown of the whole VM, not of one configuration, and interleaving made both sizes absorb it equally. The 1st and 2nd run of a round differ by less than the noise (all non-warm-up runs: read/write 855.5 vs 860.8 ns, mmap 19.4 vs 18.9 ns), so the execution order does not bias the comparison. mmap 10M warmed up for longer than $K = 2$ rounds (rounds 1–8 are elevated). K was fixed in advance and was not changed; the median-based difference (19.2 %) shows the conclusion does not depend on it.

3.4 Intrusiveness of the measuring tools

Tool vs plain `/usr/bin/time`	**read/write**	**mmap**	**Conclusion**
`perf stat` (10M, N = 20 pairs)	1.9335 → 1.9418 s, +0.43 %, p = 0.70	1.8285 → 1.9230 s, +5.2 %, p = 0.18	not detectable at N = 20
`strace -c` (1M, 3 runs)	0.19 → 79.1 s, ≈ 410×	0.40 → 0.87 s, ≈ 2.2×	huge for syscall-heavy code

Table 10: Overhead of the measuring tools (`data/intrusive.csv`, `results/strace_overhead.txt`)

`strace` stops the traced process at every syscall entry and exit (ptrace) and switches to the tracer: ≈ 874 k syscalls × ≈ 90 µs instead of ≈ 0.2 µs each. mmap makes only ≈ 7 syscalls per pass (≈ 3 500 in this run, ≈ 130 µs of extra time each), so the absolute overhead is small. Intrusiveness therefore depends not only on the tool but on how the measured program interacts with the OS. This is why the final series was measured only with `/usr/bin/time` and `date`.

4 Conclusion

1. **Answer to the variant.** With the page cache and files much smaller than RAM, the size of the file does matter, even though the disk is not involved at all: 10 MiB is 7.7 % slower per node than 1 MiB with `read`/`write`, and 19.4 % slower with `mmap`. The effect comes from the CPU caches and the TLB, and it appears wherever the code that touches the data runs: in the kernel for read/write, in user space for mmap. mmap is 43–48× faster than read/write in this workload.
2. **Critical conditions.** The page cache: writing without it was ≈ 245× slower (90.5 s vs 0.37 s). Background load: +57 % wall time and 23× the spread. Warm-up after creating a file, which needed `sync` and discarded rounds. Equal work per run: without it the comparison would only show “10× more nodes”. Interleaving: shared noise events hit both sizes equally.
3. **Negligible here.** The disk model and the virtual disk stack: 0 major faults, so the disk never took part in measured runs. `perf stat` overhead, which was not detectable. The order within a round.
4. **Uncontrolled factors, stated honestly.** CPU frequency and the P/E core placement of vCPUs are decided by macOS; hardware performance counters are unavailable in the guest, so the cache/TLB explanation rests on the user/sys split rather than on direct miss counts.

5 Appendix A. Reproduction

```
cd lab/intro-exp
./build.sh                          # clang version + both programs
./gen_graphs.sh                     # graphs + sha256 (must match Table 3)
./passport.sh                       # logs/passport.txt
sync; ./snapshot.sh before_final
./collect.sh graph_traverse      62 data/final_read.csv
./collect.sh graph_traverse_mmap 62 data/final_mmap.csv
./snapshot.sh after_final
python3 analyze.py data/final_read.csv data/final_mmap.csv   # results/, figures/
./intrusive.sh 22 data/intrusive.csv
```

Files: `build.sh`, `gen_graphs.sh`, `passport.sh`, `snapshot.sh`, `collect.sh`, `intrusive.sh`, `analyze.py`; data in `data/` (pilot, final, intrusiveness); logs in `logs/` (baseline, perf/stress-ng, passport, snapshots); tables in `results/`; figures in `figures/`.

6 Appendix B. Removed outliers (1.5 × IQR)

Configuration	Removed (round: wall, s)	Count	Common feature
read/write 1M	20: 2.020; 52: 1.895; 53: 1.913; 54: 1.867	4	user and sys both elevated
read/write 10M	11: 2.255; 16: 2.067; 19: 2.135; 41: 2.242; 54: 2.175	5	sys +6–19 %
mmap 1M	21: 1.701; 23: 1.684; 40: 1.772	3	user and sys both elevated
mmap 10M	3: 2.054; 6: 2.432; 7: 2.388; 8: 2.108; 14: 2.282; 27: 2.296; 37: 2.228	7	mostly early rounds

Table 11: Outliers (`results/outliers.csv`)

7 Appendix C. Collection script `collect.sh`

```
#!/usr/bin/env bash
# Usage: ./collect.sh <graph_traverse|graph_traverse_mmap> <N> <out.csv>
set -euo pipefail
cd "$(dirname "$0")"
PROG=$1; N=$2; OUT=$3
CORE=2
if [ "$PROG" = "graph_traverse" ]; then
  declare -A ITER=( [1M]=50   [10M]=5 )
else
  declare -A ITER=( [1M]=2000 [10M]=200 )
fi
declare -A NODES=( [1M]=43689 [10M]=436905 )
sudo renice -n -5 -p $$ > /dev/null
for s in 1M 10M; do                                  # cache warm-up, untimed
  taskset -c $CORE ./out/$PROG --write ${ITER[$s]} graphs/graph-rand-$s.bin 2>/dev/null
done
echo
"timestamp,program,size,flags,iter,nodes,core,round,pos,wall_s,user_s,sys_s,minflt,majflt,vcs,ics"
> "$OUT"
for round in $(seq 1 "$N"); do
  pos=0
  for s in $(printf "1M\n10M\n" | shuf); do           # random order in every round
    pos=$((pos + 1))
    t0=$(date +%s%N)
    /usr/bin/time -f "%U,%S,%R,%F,%w,%c" -o /tmp/collect_time.txt \
      taskset -c $CORE ./out/$PROG --write ${ITER[$s]} graphs/graph-rand-$s.bin 2>/dev/null
    t1=$(date +%s%N)
    wall=$(awk -v a="$t0" -v b="$t1" 'BEGIN { printf "%.6f", (b - a) / 1e9 }')
    echo "$(date +%s),$PROG,$s,--write,${ITER[$s]},${NODES[$s]},$CORE,$round,$pos,$wall,
$(cat /tmp/collect_time.txt)" >> "$OUT"
  done
  echo "round $round/$N done"
done
```